

Contenido

1. Introducción al refactoring
2. Un ejemplo: Renta de videos
3. Otros aspectos importantes del refactoring
4. Olores
 1. Olores dentro de las clases
 2. Olores entre clases

1. Introducción al refactoring

- El refactoring consiste en una serie de pasos pequeños, cada uno de los cuales cambia la estructura interna de un programa sin cambiar su comportamiento externo.
- Para verificar que no haya cambios en el comportamiento externo se utilizan:
 - Pruebas
 - Herramientas para el análisis formal de código

Introducción al refactoring (2)

- De donde viene:
 - Ward Cunningham y Kent Beck
 - El estilo Smalltalk
 - Ralph Johnson, University of Illinois at Urbana-Champaign
 - La tesis de Bill Opdyke
 - John Brant y Don Roberts
 - “The refactoring browser”

Introducción al refactoring (3)

- Metáfora básica:
 - Iniciar con el código existente y mejorarlo
- Definición formal:
 - El proceso de cambiar un sistema de software tal que:
 - El comportamiento externo no cambia
 - La estructura interna del sistema se mejora
- Meta:
 - Hacer el software mas fácil de entender y de modificar

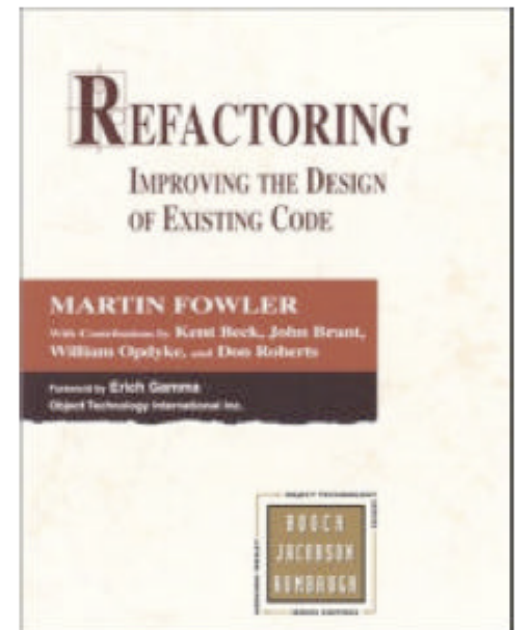
Introducción al refactoring (4)

- Los administradores odian el refactoring
 - “Si los programadores dedican mas tiempo a limpiar el código, dedican menos tiempo a implementar la funcionalidad”
- También se puede mejorar el código de manera significativa:
 - Reduciendo el código muy parecido
 - Mejorando el entendimiento, comunicando la intención
 - Mejorando la cohesión
 - Mejorando la parametrización, mantenimiento, flexibilidad, abstracción, eficiencia, etc.

Introducción al refactoring (5)

- La referencia obligada:

Fowler, M. **Refactoring: Improving the design of existing code**. Addison-Wesley, 1999.



Introducción al refactoring (6)

- El libro contiene ideas que han estado en el ambiente desde hace mucho tiempo.
- Sobre todo, este libro es un catalogo de transformaciones que se pueden aplicar a nuestro código.
 - 22 “(malos) olores” (antipatrones)
 - 72 “refactorizaciones” (que hacer al encontrar los malos olores)

Introducción al refactoring (7)

- Cuando se debe hacer refactoring ? Que tan seguido ? Cuanto tiempo se debe dedicar ?
 - No es algo a lo que se dediquen dos semanas cada seis meses ... mas bien, se hace mientras se desarrolla.
 - Se hace refactoring cuando se reconoce algún signo de alerta (olor) y se sabe que hacer.

Introducción al refactoring (8)

- La regla de tres:
 - La primera vez que codifique una tarea, solo hágala.
 - La segunda vez que codifique la misma idea, aguante y codifíquela de nuevo.
 - La tercera vez que codifique la misma idea, es tiempo de hacer refactoring.

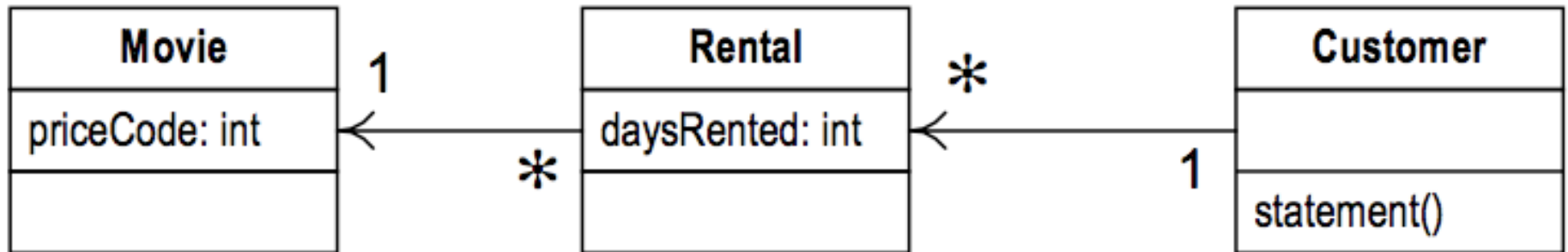
Introducción al refactoring (9)

- Antes de hacer refactoring:
 - Recuerde que el propósito es “cambiar la estructura interna preservando la semántica”
 - Cómo sabremos que se preserva la semántica ?
 - Al hacer cambios se pueden introducir errores.
 - Se debe tener un conjunto sólido de pruebas para las secciones a las que se haga refactoring.

2. Un ejemplo: renta de videos

- Considere un programa que calcule e imprima una factura de los cargos de un cliente en una tienda de renta de videos.
 - Las películas se les rentan a los clientes por un numero de días.
 - Hay tres tipos de películas: normales, para niños y estrenos.
 - Los clientes acumulan puntos por rentas frecuentes.

Diagrama de clases inicial



Clase Movie (Película)

```
public class Movie {  
    public static final int REGULAR = 0;  
    public static final int NEW_RELEASE = 1;  
    public static final int CHILDREN = 2;  
  
    private String _title;  
    private int _priceCode;  
  
    public Movie(String title, int priceCode) {  
        _title = title;  
        _priceCode = priceCode;  
    }  
  
    public int getPriceCode() {  
        return _priceCode;  
    }  
  
    public void setPriceCode(int arg) {  
        _priceCode = arg;  
    }  
  
    public String getTitle() {  
        return _title;  
    }  
}
```

Clase Rental (Renta)

```
public class Rental {  
    private Movie _movie;  
    private int _daysRented;  
  
    public Rental(Movie movie, int daysRented) {  
        _movie = movie;  
        _daysRented = daysRented;  
    }  
  
    public int getDaysRented() {  
        return _daysRented;  
    }  
  
    public Movie getMovie() {  
        return _movie;  
    }  
}
```

Clase Customer (Cliente)

```
class Customer {  
    private String _name;  
    private java.util.Vector _rentals = new java.util.Vector();  
  
    public Customer(String name) {  
        _name = name;  
    }  
  
    public void addRental(Rental arg) {  
        _rentals.addElement(arg);  
    }  
  
    public String getName() {  
        return _name;  
    }  
}
```

Clase Customer (2)

```
public String statement() {
    double totalAmount = 0;
    int frequentRenterPoints= 0;
    Enumeration rentals = _rentals.elements();
    String result = "Rental Record for " + getName() + "\n";

    while (rentals.hasMoreElements()) {
        double thisAmount= 0;
        Rental each = (Rental) rentals.nextElement();

        // Determine amounts for each line of movies
        switch (each.getMovie().getPriceCode()) {
            case Movie.REGULAR:
                thisAmount += 2;
                if (each.getDaysRented() > 2)
                    thisAmount+= (each.getDaysRented() - 2) * 1.5;
                break;
            case Movie.NEW_RELEASE:
                thisAmount += each.getDaysRented() * 3;
                break;
            case Movie.CHILDRENS:
                thisAmount += 1.5;
                if (each.getDaysRented() > 3)
                    thisAmount += (each.getDaysRented() - 3) * 1.5;
                break;
        }
    }
}
```


Clase Customer (3)

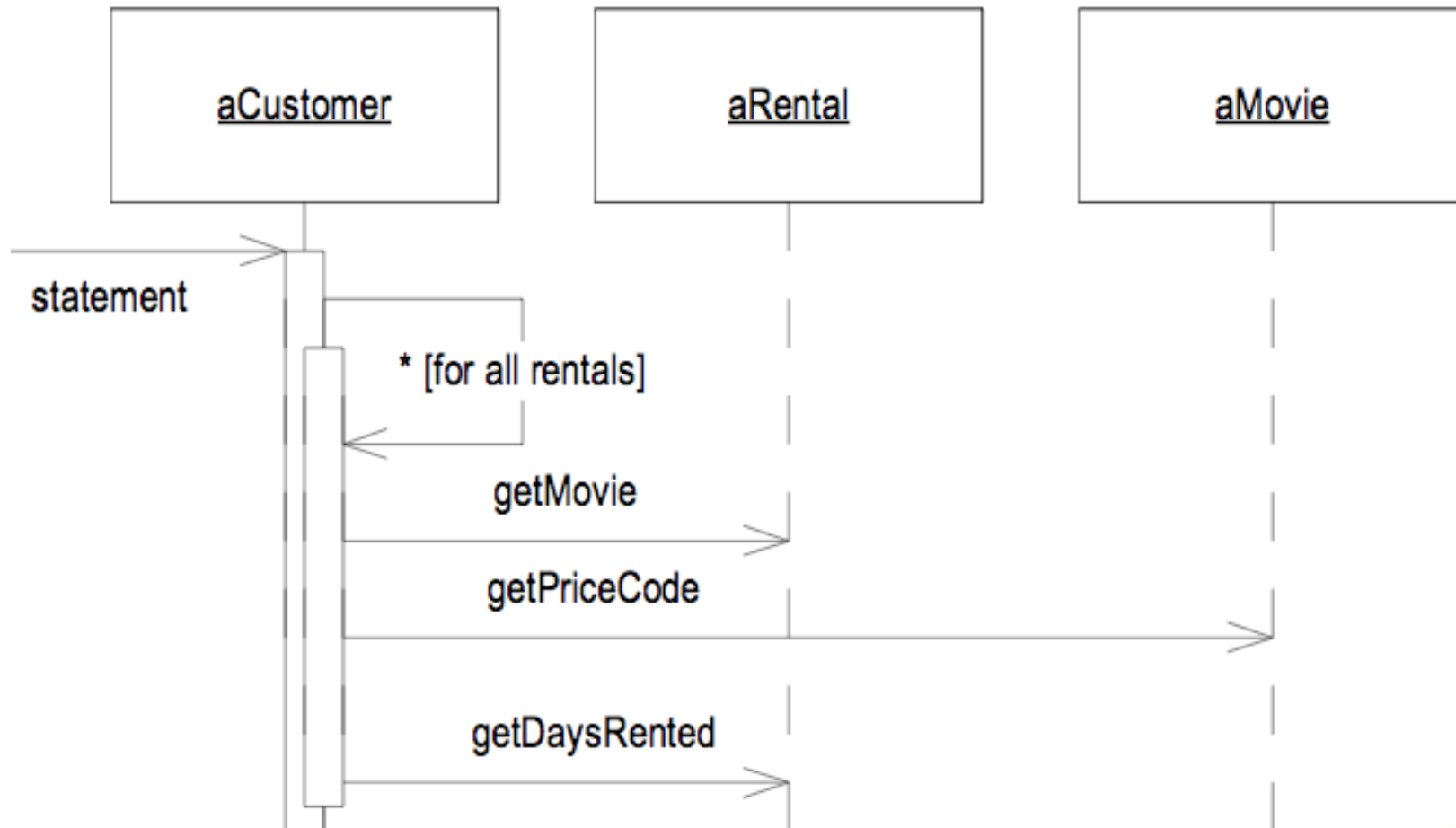
```
// Add frequent renter points
frequentRenterPoints++;

// Add bonus for a two-day new release rental
if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE) &&
    each.getDaysRented() > 1)
    frequentRenterPoints++;

// Show figures for this rental
result += "\t" + each.getMovie().getTitle() + "\t" +
        String.valueOf(thisAmount) + "\n";
totalAmount += thisAmount;
}

// Add footer lines
result += "Amount owed is " + String.valueOf(totalAmount) + "\n";
result += "You earned " + String.valueOf(frequentRenterPoints) +
        " frequent renter points";
return result;
}
}
```

Interacción para el método *statement*



Algunos comentarios

- Cuál es la impresión sobre este programa ?
- Porque el método *statement()* de la clase *Customer* hace tanto ?
- Qué pasaría si estas clases fueran parte de un sistema más complejo ?
- Independientemente de los comentarios anteriores, el programa funciona.
- Entonces, solo se hacen juicios estéticos ?
 - Así es, hasta que queremos cambiar el código.
- Los sistemas con un diseño pobre son difíciles de cambiar:
 - Porque ? Porque no sabemos donde hacer los cambios.
 - Consecuentemente, hay una posibilidad grande de introducir errores.

Cambios en los requisitos

- Producir una versión html de la factura.
- Las clasificaciones de las películas cambiarán pronto, además de las reglas para hacer los cargos y para los puntos por rentas frecuentes.

Extracción de método (extract method)

- Se tiene un fragmento de código que puede ser agrupado.
- Convierta el fragmento de código en un método cuyo nombre explique el propósito del mismo.

```
void printOwing() {  
    printBanner();  
  
    //print details  
    System.out.println ("name:" + _name);  
    System.out.println ("amount" + getOutstanding());  
}
```

```
void printOwing() {  
    printBanner();  
    printDetails(getOutstanding());  
}
```

```
void printDetails (double outstanding) {  
    System.out.println ("name:" + _name);  
    System.out.println ("amount" + outstanding);  
}
```

Candidato para extracción

```
public String statement() {
    double totalAmount = 0;
    int frequentRenterPoints= 0;
    Enumeration rentals = _rentals.elements();
    String result = "Rental Record for " + getName() + "\n";

    while (rentals.hasMoreElements()) {
        double thisAmount= 0;
        Rental each = (Rental) rentals.nextElement();

        // Determine amounts for each line of movies
        switch (each.getMovie().getPriceCode()) {
            case Movie.REGULAR:
                thisAmount += 2;
                if (each.getDaysRented() > 2)
                    thisAmount+= (each.getDaysRented() -2) * 1.5;
                break;
            case Movie.NEW_RELEASE:
                thisAmount += each.getDaysRented() * 3;
                break;
            case Movie.CHILDRENS:
                thisAmount += 1.5;
                if (each.getDaysRented() > 3)
                    thisAmount += (each.getDaysRented() - 3) * 1.5;
                break;
        }
    }
}
```

Pasos para la extracción de un método

- Crear el método de acuerdo a la intención del código.
- Copiar el código extraído.
- Buscar variables locales y parámetros.
 - Definir los parámetros
 - Definir el valor de regreso
 - Declarar dentro del método
- Compilar.
- Reemplazar el fragmento de código con una invocación al nuevo método.
- Compilar y probar.

Extracción del calculo de la cantidad a pagar

```
private double amountFor(Rental each) {  
    double thisAmount = 0;  
  
    switch (each.getMovie().getPriceCode()) {  
        case Movie.REGULAR:  
            thisAmount += 2;  
  
            if (each.getDaysRented() > 2)  
                thisAmount += (each.getDaysRented() - 2) * 1.5;  
  
            break;  
        case Movie.NEW_RELEASE:  
            thisAmount += each.getDaysRented() * 3;  
            break;  
        case Movie.CHILDRENS:  
            thisAmount += 1.5;  
  
            if (each.getDaysRented() > 3)  
                thisAmount += (each.getDaysRented() - 3) * 1.5;  
  
            break;  
    }  
  
    return thisAmount;  
}
```


El método *statement()* después de la extracción

```
public String statement() {
    double totalAmount = 0;
    int frequentRenterPoints= 0;
    Enumeration rentals = _rentals.elements();
    String result = "Rental Record for " + getName() + "\n";

    while (rentals.hasMoreElements()) {
        double thisAmount= 0;
        Rental each = (Rental) rentals.nextElement();

        thisAmount = amountFor(each);

        // Add frequent renter points
        frequentRenterPoints++;

        // Add bonus for a two-day new release rental
        if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE) && each.getDaysRented() > 1)
            frequentRenterPoints++;

        // Show figures for this rental
        result += "\t" + each.getMovie().getTitle() + "\t" + String.valueOf(thisAmount) + "\n";
        totalAmount += thisAmount;
    }

    // Add footer lines
    result += "Amount owed is " + String.valueOf(totalAmount) + "\n";
    result += "You earned " + String.valueOf(frequentRenterPoints) + " frequent renter points";
    return result;
}
```

Cambio de los nombres de las variables

```
private double amountFor(Rental aRental) {
    double result = 0;

    switch (aRental.getMovie().getPriceCode()) {
        case Movie.REGULAR:
            result += 2;

            if (aRental.getDaysRented() > 2)
                result += (aRental.getDaysRented() - 2) * 1.5;

            break;
        case Movie.NEW_RELEASE:
            result += aRental.getDaysRented() * 3;
            break;
        case Movie.CHILDRENS:
            result += 1.5;

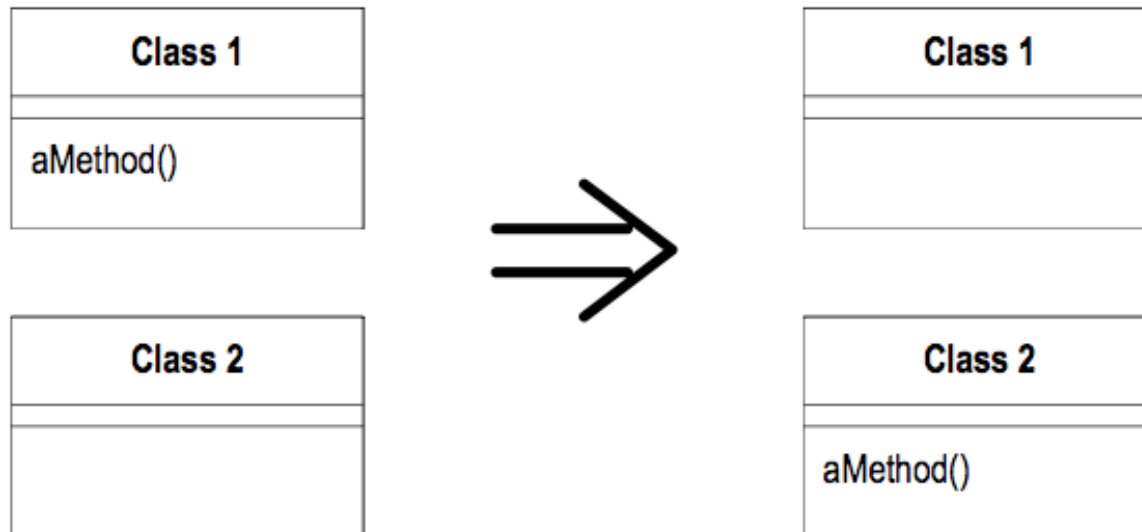
            if (aRental.getDaysRented() > 3)
                result += (aRental.getDaysRented() - 3) * 1.5;

            break;
    }

    return result;
}
```

Movimiento de un método (move method)

- Un método es, o será, utilizado mas por otra clase que por su clase misma.
- Crear un nuevo método con un cuerpo similar, en la clase donde se utiliza mas.
- Modifique el método viejo para que sea una delegación simple o remuévalo.



Pasos para el movimiento de un método

- Declare el método en la clase objetivo.
- Copie y ajuste el código.
- Agregue una referencia del objeto fuente al objetivo.
- Convierta el método original en un método delegado.
 - **amountOf(Rental each) { return each.charge();}**
 - Verifique los métodos sobrecargados.
- Compile y pruebe
- Encuentre a todos los usuarios del método.
- Ajústelos para que invoquen al método objetivo.
- Remueva el método original.
- Compile y pruebe.

Moviendo amount() a la clase Rental

```
class Rental {
    . . .
    double getCharge() {
        double result = 0;

        switch (getMovie().getPriceCode()) {
            case Movie.REGULAR:
                result += 2;

                if (getDaysRented() > 2)
                    result += (getDaysRented() - 2) * 1.5;

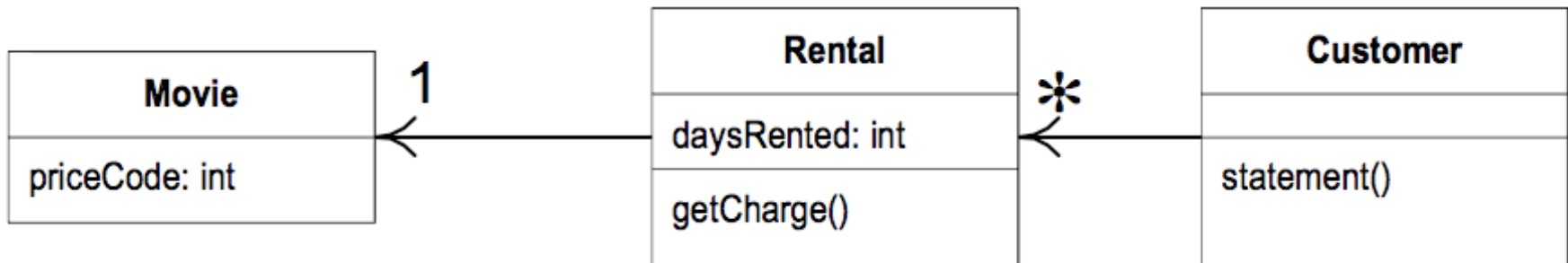
                break;
            case Movie.NEW_RELEASE:
                result += getDaysRented() * 3;
                break;
            case Movie.CHILDRENS:
                result += 1.5;

                if (getDaysRented() > 3)
                    result += (getDaysRented() - 3) * 1.5;

                break;
        }

        return result;
    }
}
```

Moviendo amount() a la clase Rental (2)



El método *statement()* modificado

```
class Customer { . . .
    public String statement() {
        double totalAmount = 0;
        int frequentRenterPoints= 0;
        Enumeration rentals = _rentals.elements();
        String result = "Rental Record for " + getName() + "\n";

        while (rentals.hasMoreElements()) {
            double thisAmount= 0;
            Rental each = (Rental) rentals.nextElement();
            thisAmount = each.getCharge();

            // Add frequent renter points
            frequentRenterPoints++;

            // Add bonus for a two-day new release rental
            if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE) && each.getDaysRented() > 1)
                frequentRenterPoints++;

            // Show figures for this rental
            result += "\t" + each.getMovie().getTitle() + "\t" + String.valueOf(thisAmount) + "\n";
            totalAmount += thisAmount;
        }

        // Add footer lines
        result += "Amount owed is " + String.valueOf(totalAmount) + "\n";
        result += "You earned " + String.valueOf(frequentRenterPoints) + " frequent renter points";
        return result;
    }
}
```

Problemas con las variables temporales

```
class Customer { . . .
    public String statement() {
        double totalAmount = 0;
        int frequentRenterPoints= 0;
        Enumeration rentals = _rentals.elements();
        String result = "Rental Record for " + getName() + "\n";

        while (rentals.hasMoreElements()) {
            double thisAmount= 0;
            Rental each = (Rental) rentals.nextElement();
            thisAmount = each.getCharge();

            // Add frequent renter points
            frequentRenterPoints++;

            // Add bonus for a two-day new release rental
            if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE) && each.getDaysRented() > 1)
                frequentRenterPoints++;

            // Show figures for this rental
            result += "\t" + each.getMovie().getTitle() + "\t" + String.valueOf(thisAmount) + "\n";
            totalAmount += thisAmount;
        }

        // Add footer lines
        result += "Amount owed is " + String.valueOf(totalAmount) + "\n";
        result += "You earned " + String.valueOf(frequentRenterPoints) + " frequent renter points";
        return result;
    }
}
```


Reemplazar la variable temporal con una consulta

- Se esta utilizando una variable temporal para mantener el resultado de una expresion.
- Extraer la expresion en un método.
- Reemplazar todas las referencias a la variable temporal con la expresion.
- El nuevo método podra ser utilizado en otros métodos.

Pasos para reemplazar variables temporales por consultas

- Encuentre la variable temporal con una asignación sencilla.
- Extraiga la parte de la derecha de la asignación.
- Reemplace las referencias a la variable temporal con un método nuevo.
- Remueva la declaración y la asignación de la variable temporal.
- Compile y pruebe.

Remoción de la variable thisAmount

```
public String statement() {
    double totalAmount = 0;
    int frequentRenterPoints= 0;
    Enumeration rentals = _rentals.elements();
    String result = "Rental Record for " + getName() + "\n";

    while (rentals.hasMoreElements()) {
        Rental each = (Rental) rentals.nextElement();

        // Add frequent renter points
        frequentRenterPoints++;

        // Add bonus for a two-day new release rental
        if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE) && each.getDaysRented() > 1)
            frequentRenterPoints++;

        // Show figures for this rental
        result += "\t" + each.getMovie().getTitle() + "\t" + String.valueOf(each.getCharge()) +
"\n";
        totalAmount += each.getCharge();
    }

    // Add footer lines
    result += "Amount owed is " + String.valueOf(totalAmount) + "\n";
    result += "You earned " + String.valueOf(frequentRenterPoints) + " frequent renter points";
    return result;
}
```

Extracción y movimiento del método frequentRenterPoints()

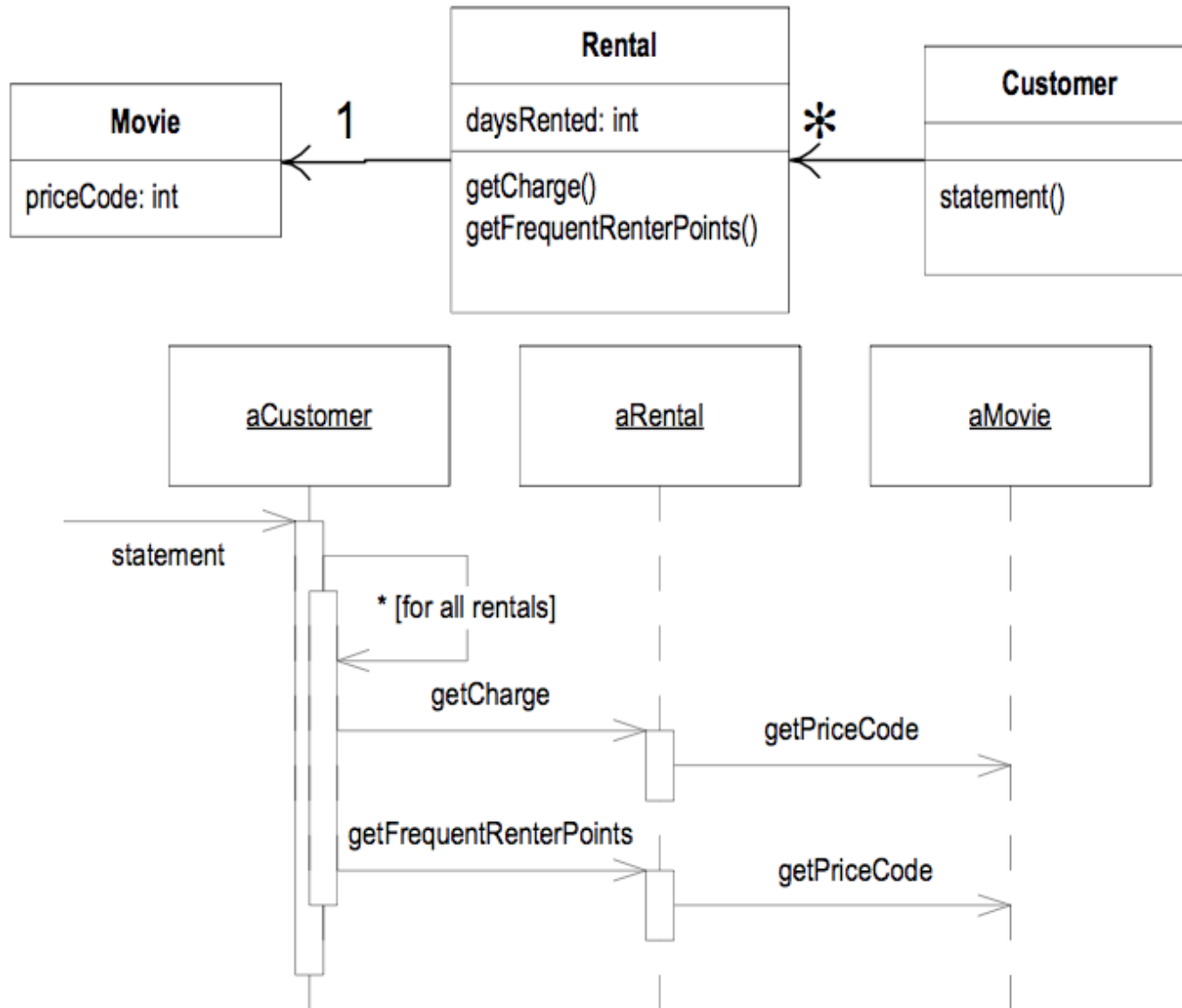
```
class Customer { . . .
    public String statement() {
        double totalAmount = 0;
        int frequentRenterPoints= 0;
        Enumeration rentals = _rentals.elements();
        String result = "Rental Record for " + getName() + "\n";

        while (rentals.hasMoreElements()) {
            Rental each = (Rental) rentals.nextElement();
            frequentRenterPoints += each.getFrequentRenterPoints();

            // Show figures for this rental
            result += "\t" + each.getMovie().getTitle() + "\t"
                + String.valueOf(each.getCharge())
                + "\n";
            totalAmount += each.getCharge();
        }

        // Add footer lines
        result += "Amount owed is " + String.valueOf(totalAmount) + "\n";
        result += "You earned " + String.valueOf(frequentRenterPoints) +
            " frequent renter points";
        return result;
    }
}
```

Despues de mover getCharge() y frequentRenterPoints()



Mas temporales por mover

```
class Customer { . . .
    public String statement() {
        double totalAmount = 0;
        int frequentRenterPoints= 0;
        Enumeration rentals = _rentals.elements();
        String result = "Rental Record for " + getName() + "\n";

        while (rentals.hasMoreElements()) {
            Rental each = (Rental) rentals.nextElement();
            frecuentRenterPoints += each.getFrecuentRenterPoints();

            // Show figures for this rental
            result += "\t" + each.getMovie().getTitle() + "\t" +
String.valueOf(each.getCharge())
                + "\n";
            totalAmount += each.getCharge();
        }

        // Add footer lines
        result += "Amount owed is " + String.valueOf(totalAmount) + "\n";
        result += "You earned " + String.valueOf(frequentRenterPoints) +
            " frequent renter points";
        return result;
    }
}
```

Los nuevos métodos

```
class Customer { . . .
    private double getTotalCharge() {
        double result = 0;
        Enumeration rentals = _rentals.elements();

        while (rentals.hasMoreElements()) {
            Rental each = (Rental) rentals.nextElement();
            result += each.getCharge();
        }

        return result;
    }

    private double getTotalFrecuentRenterPoints() {
        int result = 0;
        Enumeration rentals = _rentals.elements();

        while (rentals.hasMoreElements()) {
            Rental each = (Rental) rentals.nextElement();
            result += each.getFrecuentRenterPoints();
        }

        return result;
    }
}
```

Remoción de las variables temporales

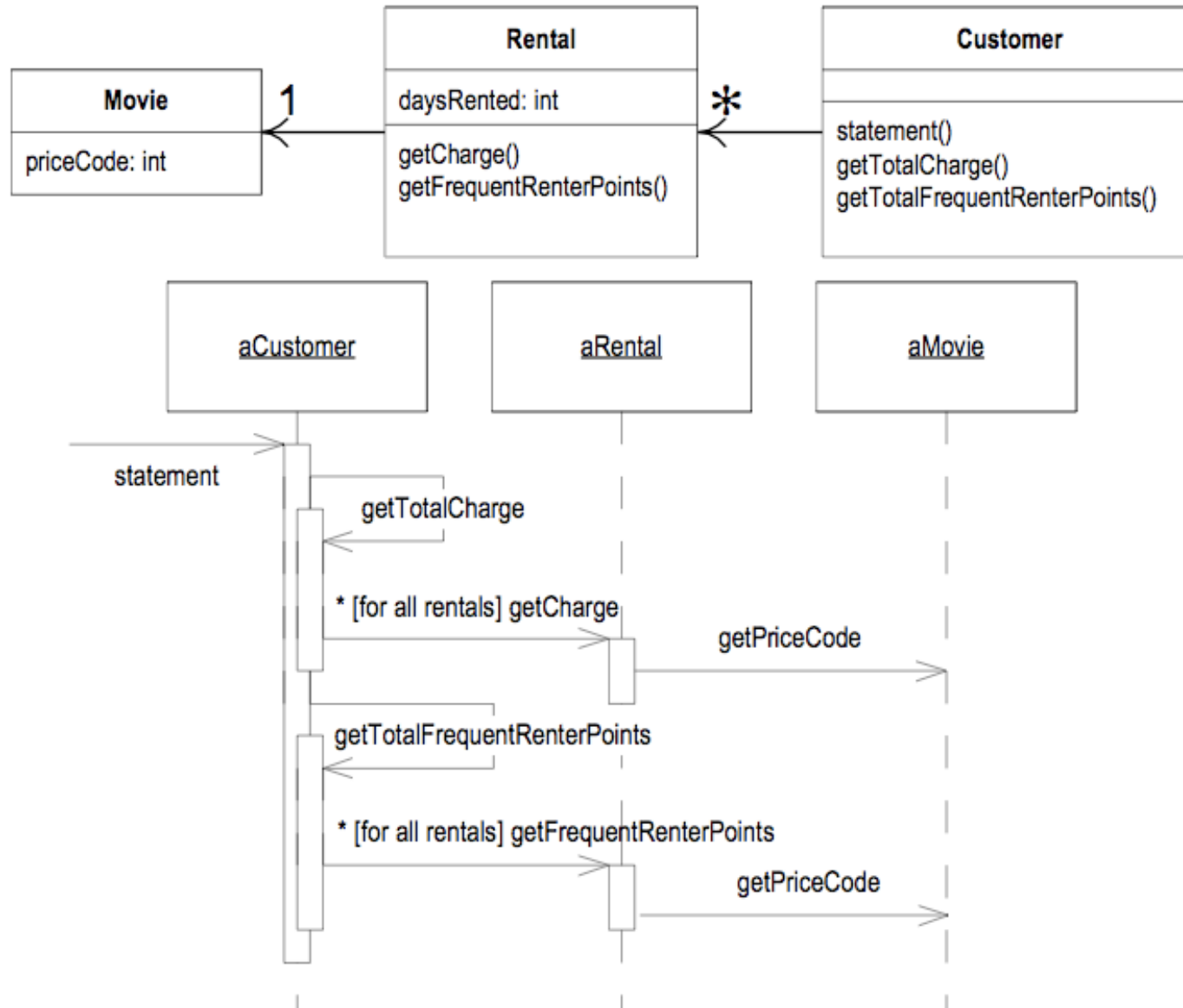
```
class Customer { . . .
    public String statement() {
        Enumeration rentals = _rentals.elements();
        String result = "Rental Record for " + getName() + "\n";

        while (rentals.hasMoreElements()) {
            Rental each = (Rental) rentals.nextElement();

            // Show figures for this rental
            result += "\t" + each.getMovie().getTitle() + "\t" +
                String.valueOf(each.getCharge()) + "\n";
        }

        // Add footer lines
        result += "Amount owed is " + String.valueOf(getTotalCharge()) + "\n";
        result += "You earned " +
            String.valueOf(getTotalFrequentRenterPoints())
                + " frequent renter points";
        return result;
    }
}
```


Despues de reemplazar los totales



El nuevo método htmlStatement()

```
public String htmlStatement() {
    Enumeration rentals = _rentals.elements();
    String result = "<H1>Rentals for <EM>" + getName()
        + "</EM></H1><P>\n";

    while (rentals.hasMoreElements()) {
        Rental each = (Rental) rentals.nextElement();

        // Show figures for this rental
        result += each.getMovie().getTitle() + ": " +
            String.valueOf(each.getCharge()) + "<BR>\n";
    }

    // Add footer lines
    result += "<P> You owe <EM> " + String.valueOf(getTotalCharge())
        + "</EM> <P>\n";
    result += "On this rental you earned <EM>" +
        String.valueOf(getTotalFrequentRenterPoints()) +
        "</EM> frequent renter points<P>";
    return result;
}
```

El método getCharge() actual

```
class Rental {
    double getCharge() {
        double result = 0;

        switch (getMovie().getPriceCode()) {
            case Movie.REGULAR:
                result += 2;

                if (getDaysRented() > 2)
                    result += (getDaysRented() - 2) * 1.5;

                break;
            case Movie.NEW_RELEASE:
                result += getDaysRented() * 3;
                break;
            case Movie.CHILDRENS:
                result += 1.5;

                if (getDaysRented() > 3)
                    result += (getDaysRented() - 3) * 1.5;

                break;
        }

        return result;
    }
}
```

Movimiento de getCharge() a la clase Movie

```
class Rental { . . .
    double getCharge() {
        return _movie.getCharge(_daysRented);
    }
}

class Movie { . . .
    double getCharge(int daysRented) {
        double result = 0;

        switch (getPriceCode()) {
            case Movie.REGULAR:
                result += 2;

                if (daysRented > 2)
                    result += (daysRented - 2) * 1.5;

                break;
            case Movie.NEW_RELEASE:
                result += daysRented * 3;
                break;
            case Movie.CHILDRENS:
                result += 1.5;

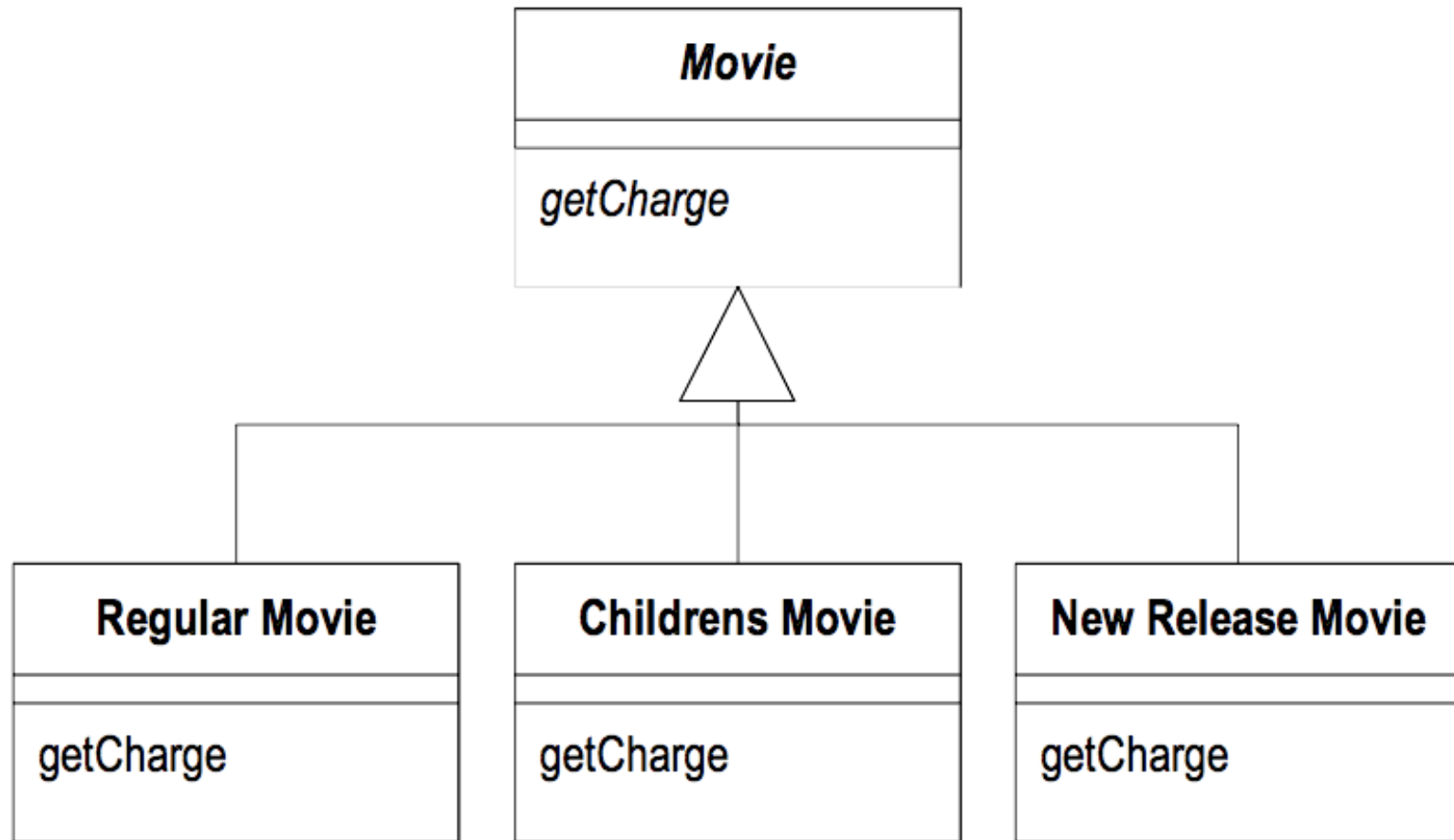
                if (daysRented > 3)
                    result += (daysRented - 3) * 1.5;

                break;
        }

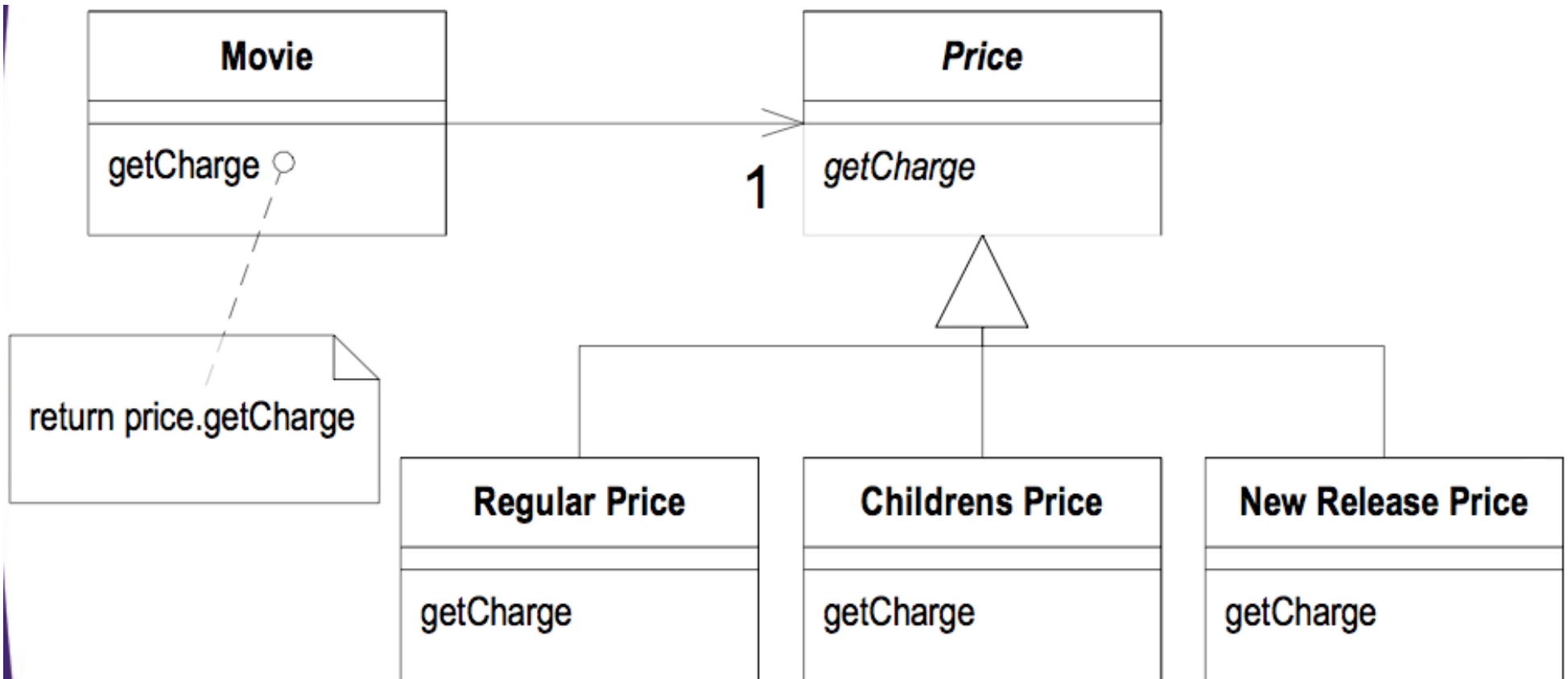
        return result;
    }
}
```

- Se hace lo mismo con frequentRenterPoints()

Considere la herencia

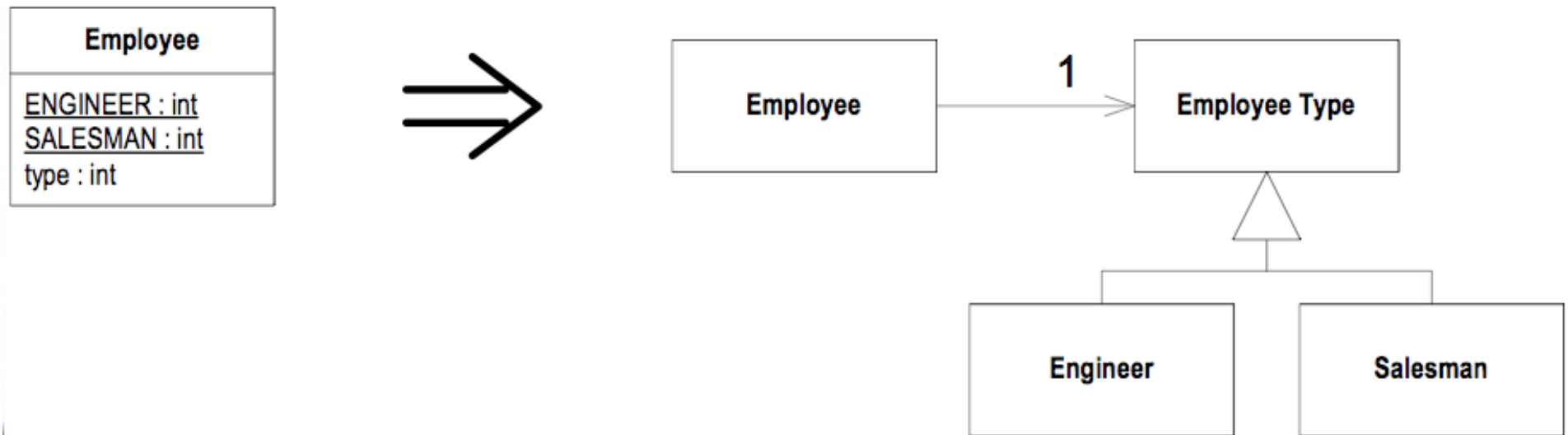


Utilice el patrón de estado



Reemplazo de código de tipo a través del patrón de estado/estrategia

- Se tiene un código de tipo que afecta el comportamiento de una clase, pero no se pueden utilizar subclases.
- Reemplazar el código de tipo con un objeto de estado.



Pasos para reemplazar un código de tipo a través del patrón de estado/estrategia

- Cree una nueva clase de estado para el código de tipo.
- Agregue subclases de la clase estado para cada tipo.
- Cree una consulta abstracta en la superclase para regresar el tipo y sobrescriba en las subclases para regresar el tipo correcto.
- Compile.
- Cree un campo en la clase vieja para el objeto de estado.
- Cambie la consulta del tipo para delegar hacia el objeto de estado.
- Cambie los métodos que asignan los tipos para que asignen instancias de las subclases.
- Compile y pruebe.

Códigos de precio en la jerarquía de precios

```
abstract class Price {  
    abstract int getPriceCode();  
}
```

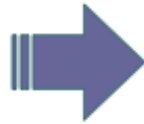
```
class ChildrensPrice extends Price {  
    int getPriceCode() {  
        return Movie.CHILDRENS;  
    }  
}
```

```
class NewReleasePrice extends Price {  
    int getPriceCode() {  
        return Movie.NEW_RELEASE;  
    }  
}
```

```
class RegularPrice extends Price {  
    int getPriceCode() {  
        return Movie.REGULAR;  
    }  
}
```

Cambio de accesadores en la clase Movie

```
public int getPriceCode() {  
    return _priceCode;  
}  
  
public setPriceCode(int arg) {  
    _priceCode = arg;  
}  
  
private int _priceCode;
```

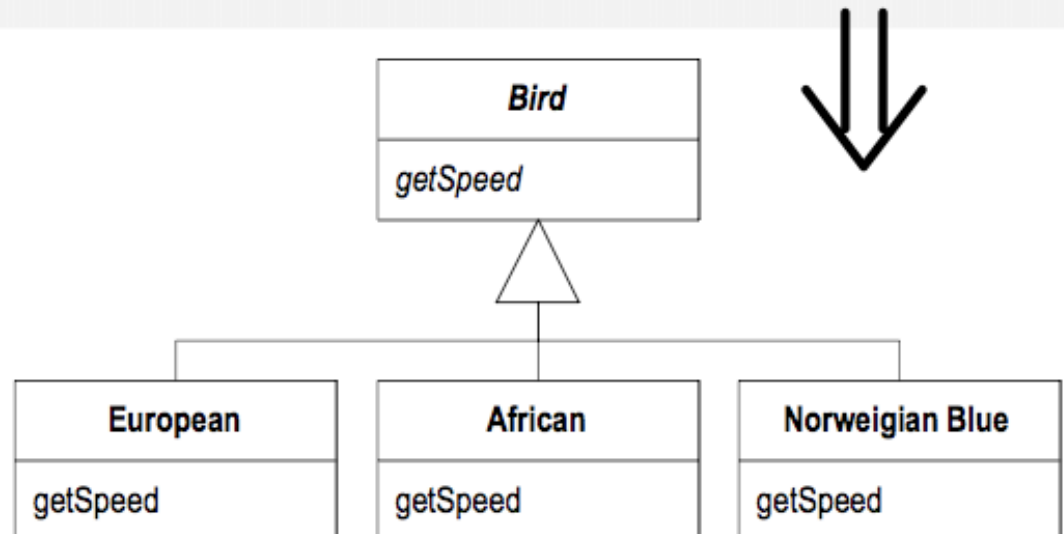


```
public int getPriceCode() {  
    return _price.getPriceCode;  
}  
  
public setPriceCode(int arg) {  
    switch(arg) {  
        case REGULAR:  
            _price = new RegularPrice();  
            break;  
        case CHILDRENS:  
            _price = new ChildrensPrice();  
            break;  
        case NEW_RELEASE:  
            _price = new NewReleasePrice();  
            break;  
        default:  
            throw new IllegalArgumentException  
                ("Incorrect price code");  
    }  
}  
  
private Price _price;
```

Reemplazo de condicionales con polimorfismo

- Se tiene un condicional que selecciona diferente comportamiento dependiendo del tipo de objeto.
- Mover cada rama del condicional a un método sobrescrito en una subclase.
- Hacer abstracto el método original.

```
double getSpeed() {  
    switch (_type) {  
        case EUROPEAN:  
            return getBaseSpeed();  
        case AFRICAN:  
            return getBaseSpeed() - getLoadFactor() * _numberOfCoconuts;  
        case NORWEIGIAN_BLUE:  
            return (_isNailed) ? 0 : getBaseSpeed(_voltage);  
    }  
    throw new RuntimeException ("Should be unreachable");  
}
```



Pasos para reemplazar un condicional con polimorfismo

- Mueva el enunciado *switch* a la superclase de la jerarquía.
- Copie una rama del enunciado *case* a la subclase.
- Compile y pruebe.
- Repita para las otras ramas.
- Reemplace el enunciado *case* con un método abstracto.

Movimiento de getCharge() a la clase Price

```
class Price { . . .
    double getCharge(int daysRented) {
        double result = 0;

        switch (getPriceCode()) {
            case Movie.REGULAR:
                result += 2;

                if (daysRented > 2)
                    result += (daysRented - 2) * 1.5;

                break;
            case Movie.NEW_RELEASE:
                result += daysRented * 3;
                break;
            case Movie.CHILDRENS:
                result += 1.5;

                if (daysRented > 3)
                    result += (daysRented - 3) * 1.5;

                break;
        }

        return result;
    }
}
```

```
class Movie { . . .
    double getCharge(int daysRented) {
        return _price.getCharge(daysRented);
    }
}
```

Sobreescritura de getCharge() en las subclases

```
class RegularPrice extends Price { . . .
    double getCharge(int daysRented) {
        double result = 2;

        if (daysRented > 2)
            result += (daysRented - 2) * 1.5;
        return result;
    }
}

class ChildrensPrice extends Price { . . .
    double getCharge(int daysRented) {
        double result = 1.5;

        if (daysRented > 3)
            result += (daysRented - 3) * 1.5;
        return result;
    }
}

class NewReleasePrice extends Price { . . .
    double getCharge(int daysRented) {
        return daysRented * 3;
    }
}

Class Price { . . .
    abstract double getCharge(int daysRented);
}
```

Métodos statement() similares

```
public String statement() {
    Enumeration rentals = _rentals.elements();
    String result = "Rental Record for " + getName() + "\n";

    while (rentals.hasMoreElements()) {
        Rental each = (Rental) rentals.nextElement();
        result += "\t" + each.getMovie().getTitle() + "\t" +
String.valueOf(each.getCharge())
                + "\n";
    }

    result += "Amount owed is " + String.valueOf(getTotalCharge()) + "\n";
    result += "You earned " +
String.valueOf(getTotalFrequentRenterPoints())
            + " frequent renter points";
    return result;
}
```

Métodos statement() similares

```
public String htmlStatement() {
    Enumeration rentals = _rentals.elements();
    String result = "<H1>Rentals for <EM>" + getName() + "</EM></H1><P>\n";

    while (rentals.hasMoreElements()) {
        Rental each = (Rental) rentals.nextElement();
        result += each.getMovie().getTitle() + ": " + String.valueOf(each.getCharge())
            + "<BR>\n";
    }

    result += "<P> You owe <EM> " + String.valueOf(getTotalCharge()) + "</EM> <P>\n";
    result += "On this rental you earned <EM>"
        + String.valueOf(getTotalFrequentRenterPoints())
        + "</EM> frequent renter points<P>";
    return result;
}
```


Método plantilla

- Se tienen dos métodos, en dos subclases, que llevan a cabo pasos similares en el mismo orden, sin embargo son diferentes.
- De a cada paso en los métodos la misma firma, de tal manera que los métodos originales se conviertan en el mismo.

Pasos para el método plantilla

- Tome dos métodos con estructura similar, pero variación en algunas partes.
 - Use subclases de la clase o cree una estrategia y mueva los métodos a la estrategia.
- En cada variación extraiga los métodos que tengan la misma firma, pero cuerpo diferente.
- Declare la firma de los métodos extraídos en una superclase y coloque la variación en subclases.
- Cuando se hayan removido los puntos de variación, mueva un método fuente a la superclase y remueva el otro.

Crear una estrategia para statement()

```
class Customer { . . .  
    public String statement() {  
        return new TextStatement().value(this);  
    }  
}
```

- Se hace lo mismo con `htmlStatement()`

```
class TextStatement {  
    public String value(Customer aCustomer) {  
        Enumeration rentals = aCustomer.getRentals();  
        String result = "Rental Record for " + aCustomer.getName() + "\n";  
  
        while (rentals.hasMoreElements()) {  
            Rental each = (Rental) rentals.nextElement();  
            result += "\t" + each.getMovie().getTitle() + "\t" +  
                String.valueOf(each.getCharge()) + "\n";  
        }  
  
        result += "Amount owed is " + String.valueOf(aCustomer.getTotalCharge()) + "\n";  
        result += "You earned " +  
            String.valueOf(aCustomer.getTotalFrequentRenterPoints()) +  
            " frequent renter points";  
        return result;  
    }  
}
```

Extraer las diferencias

```
class TextStatement {
    public String value(Customer aCustomer) {
        Enumeration rentals = aCustomer.getRentals();
        String result = headerString(aCustomer);

        while (rentals.hasMoreElements()) {
            Rental each = (Rental) rentals.nextElement();
            result += "\t" + each.getMovie().getTitle() + "\t" +
                String.valueOf(each.getCharge()) + "\n";
        }

        result += "Amount owed is " + String.valueOf(aCustomer.getTotalCharge()) + "\n";
        result += "You earned " +
            String.valueOf(aCustomer.getTotalFrequentRenterPoints()) +
            " frequent renter points";
        return result;
    }

    String headerString(Customer aCustomer) {
        return "Rental Record for " + aCustomer.getName() + "\n";
    }
}

class HtmlStatement { . . .
    String headerString(Customer aCustomer) {
        return "<H1> Rentals for <EM>" + aCustomer.getName() + "</EM></H1><P>\n";
    }
}
```

Continuar extrayendo

```
class TextStatement {
    public String value(Customer aCustomer) {
        Enumeration rentals = aCustomer.getRentals();
        String result = headerString(aCustomer);

        while (rentals.hasMoreElements()) {
            Rental each = (Rental) rentals.nextElement();
            result += eachRentalString(each);
        }

        result += footerString(aCustomer);
        return result;
    }

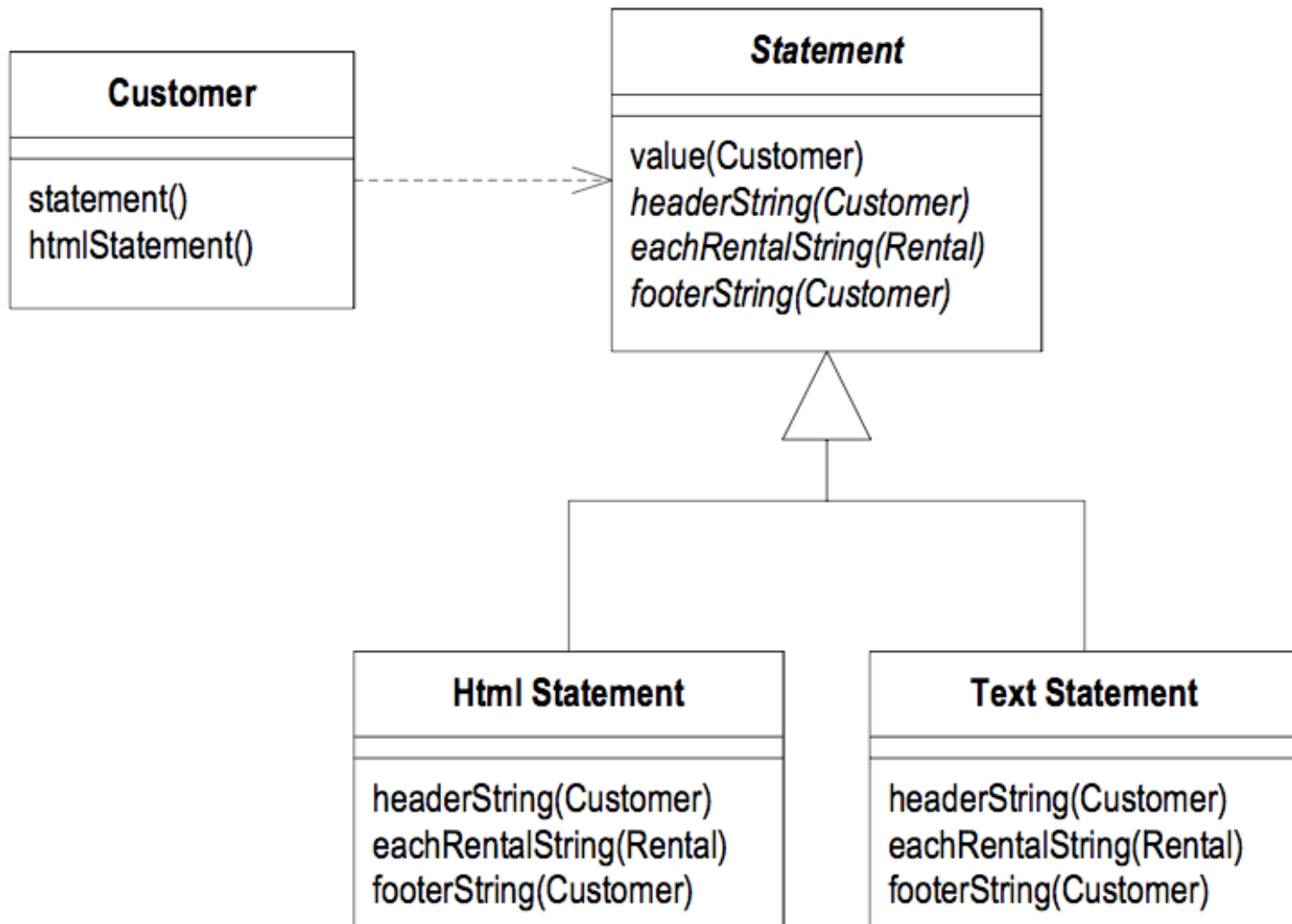
    String eachRentalString(Rental aRental) {
        return "\t" + aRental.getMovie().getTitle() + "\t" +
            String.valueOf(aRental.getCharge()) + "\n";
    }

    String footerString(Customer aCustomer) {
        return "Amount owed is " + String.valueOf(aCustomer.getTotalCharge()) +
            "\n" + "You earned " +
            String.valueOf(aCustomer.getTotalFrequentRenterPoints()) +
            " frequent renter points";
    }
}
```

Sacar el método value()

```
class Statement {  
    public String value(Customer aCustomer) {  
        Enumeration rentals = aCustomer.getRentals();  
        String result = headerString(aCustomer);  
  
        while (rentals.hasMoreElements()) {  
            Rental each = (Rental) rentals.nextElement();  
            result += eachRentalString(each);  
        }  
  
        result += footerString(aCustomer);  
        return result;  
    }  
  
    abstract String headerString(Customer aCustomer);  
    abstract String eachRentalString(Rental aRental);  
    abstract String footerString(Customer aCustomer);  
}
```

El estado de las clases



Comentarios finales del ejemplo

- En este ejemplo:
 - Se vio como mejorar un programa mal construido para facilitar:
 - Agregar nuevos servicios en la clase Customer (cliente)
 - Agregar nuevos tipos de películas
 - No se depuro durante el refactoring
 - Los pasos apropiados reducen la posibilidad de errores
 - Los pasos pequeños hacen los errores fáciles de localizar
 - Se ilustraron varios tipos de refactoring
 - Método de extracción
 - Método de movimiento
 - Reemplazo de variables temporales
 - Reemplazo de código de tipos a través del patrón de estado/estrategia
 - Reemplazo de Switch con polimorfismo
 - Método plantilla

3. Otros aspectos importantes del refactoring

- Herramientas para apoyar el refactoring
 - Basadas en las posibles transformaciones
 - Constructores de árboles de sintaxis para los programas
 - Pruebas matemáticas para asegurar que no se cambia la semántica
 - Incrustadas en otra herramienta
 - Para acelerar el proceso
 - Método de extracción
 - No se necesitan pruebas
 - Sin ciencia ficción
 - Existen herramientas para Smalltalk

Otros aspectos importantes del refactoring (2)

- La importancia de las pruebas
 - Aun con herramientas, las pruebas son importantes
 - No todos los refactoring pueden ser probados
 - Escriba pruebas según vaya escribiendo código
 - Haga las pruebas autoverificables
 - Ejecute un conjunto de pruebas con un solo comando
 - Prueba cada que compile

Otros aspectos importantes del refactoring (3)

- Problemas con el refactoring
 - Aun no se sabe lo que es
 - Migración de las bases de datos
 - Publicación de interfaces
 - Solo donde se necesite
 - Sin pruebas de funcionamiento

Otros aspectos importantes del refactoring (4)

- Decisiones de diseño
 - En el momento
 - Considere las necesidades presentes
 - Corrija el código cuando aparezcan nuevas necesidades
 - Diseño planeado
 - Considere las necesidades presentes y futuras
 - Diseñe para minimizar los cambios en el futuro
 - Corrija el código si aparecen necesidades inesperadas
 - Diseño evolucionado
 - Considere las necesidades presentes y futuras
 - Compare el costo de la flexibilidad actual contra el costo del refactoring posterior
 - Haga refactoring cuando aparezcan los cambios

Otros aspectos importantes del refactoring (5)

- Técnicas de equipo
 - Aliente la cultura del refactoring
 - Provea una base de pruebas sólida
 - Utilice programación por pares